

Git & SSH Quick Reference

For cloning private GitHub repos on macOS

1. Generate an SSH Key

Run this in your terminal (any directory is fine):

```
ssh-keygen -t ed25519 -C "your-email@example.com"
```

Press Enter to accept the default file location. Set a passphrase or press Enter twice to skip.

2. Start the SSH Agent & Add Your Key

Start the agent:

```
eval "$(ssh-agent -s)"
```

Add your key:

```
ssh-add ~/.ssh/id_ed25519
```

3. Copy Your Public Key to Clipboard

Copy the key (includes the email at the end):

```
pbcopy < ~/.ssh/id_ed25519.pub
```

To verify what was copied, you can view it with:

```
cat ~/.ssh/id_ed25519.pub
```

The key should start with ssh-ed25519 and end with your email.

4. Add the Key to GitHub

1. Go to github.com → Settings → SSH and GPG keys
2. Click **New SSH key**
3. Give it a title (e.g. "My MacBook Pro")
4. Set Key type to **Authentication Key**
5. Paste the **entire key** including the email at the end
6. Click **Add SSH key**

5. Clone Your Repo Using SSH

Use the SSH URL (not HTTPS) when cloning:

```
git clone git@github.com:username/repo-name.git
```

6. Everyday Git Commands

Command	What It Does
---------	--------------

<code>git status</code>	See what files have changed
<code>git add .</code>	Stage all changes for commit
<code>git add filename</code>	Stage a specific file
<code>git commit -m "message"</code>	Commit staged changes
<code>git push origin main</code>	Push commits to GitHub
<code>git pull origin main</code>	Pull latest changes from GitHub
<code>git log --oneline</code>	View recent commit history
<code>git branch</code>	List branches
<code>git checkout -b new-branch</code>	Create & switch to a new branch
<code>git checkout main</code>	Switch back to main branch
<code>git merge branch-name</code>	Merge a branch into current branch

7. Lovable + GitHub 2-Way Sync

When your Lovable project is connected to GitHub:

Changes in Lovable → auto-pushed to GitHub. Pull locally with:

```
git pull origin main
```

Changes made locally → push to GitHub, Lovable picks them up:

```
git add . && git commit -m "message" && git push origin main
```

8. Rolling Back Versions

First, view your commit history to find the commit you want:

```
git log --oneline
```

This shows commits like: a1b2c3d Added login page

Command	What It Does	When to Use
<code>git revert <commit-id></code>	Creates a new commit that undoes a specific commit	Safest option. Use with Lovable 2-way sync.
<code>git reset --soft HEAD~1</code>	Undoes the last commit but keeps your file changes	Undo a commit you just made, re-edit and recommit.
<code>git checkout <commit-id></code>	Temporarily view an old version (read-only)	Just want to look at an older version.
<code>git checkout main</code>	Go back to the latest version after checking out	Return to current state after viewing old version.

Important: With Lovable's 2-way sync, always use `git revert` instead of `reset --hard` or `force push`. Rewriting history can break the sync.

9. Rolling Back in Lovable's UI

Lovable has its own version history built into the editor:

1. Click the **clock icon** in the top toolbar of the Lovable editor
2. You'll see a list of every change with timestamps
3. Click on any version to **preview** what your site looked like
4. Click **Restore** to roll back to that version

***How it relates to Git:** Every Lovable edit creates a git commit. When you restore an older version in Lovable, it creates a **new commit** that reverts to that state and pushes it to GitHub automatically. Run `git pull origin main` locally to get the reverted version.*

You don't need to use `git revert` manually if you roll back through Lovable's UI — they're two different ways to do the same thing.

10. Using Claude Code with Lovable

Claude Code can edit your Lovable project locally. The workflow:

Before editing — always pull first:

```
git pull origin main
```

After Claude Code makes edits — push to sync:

```
git add . && git commit -m "message" && git push origin main
```

Lovable will automatically pick up your changes via the 2-way sync.

***Avoid editing in Lovable's AI editor and locally at the same time.** This will cause merge conflicts. Pick one at a time — use Lovable for quick UI tweaks and Claude Code for deeper code work.*

Troubleshooting

"Password authentication is not supported" — You're using HTTPS with a password. Switch to SSH (see steps above) or use a Personal Access Token instead of your password.

"Permission denied (publickey)" — Your SSH key isn't linked to GitHub. Re-run steps 3 & 4 above.

"Key is invalid" on GitHub — Make sure you copied the entire key, including the email at the end.